

# TP 5 : calculs avec les polynômes

ZFAI

Les questions marquées d'une  $\star$  sont des questions théoriques. Elles peuvent être abordées plus tard, après le TP.

L'objectif est d'implémenter différents algorithmes nous permettant de faire des calculs avec des polynômes. On rappelle qu'un polynôme est un objet qui s'écrit sous la forme :

$$P(X) = a_0 + a_1X + a_2X^2 + \cdots + a_nX^n.$$

Ainsi, on peut représenter le polynôme  $P$  en python à l'aide de la liste de ses coefficients :

$$L = [a_0, a_1, a_2, \cdots, a_n].$$

Sur les polynômes, il existe différentes opérations élémentaires :

1. la somme,
2. le produit,
3. la dérivation,
4. le calcul de  $P(a)$  pour un réel  $a$ .

Une question naturelle est alors d'implanter ces opérations en Python, pour effectuer des calculs sur les polynômes à l'aide de l'ordinateur.

## À FAIRE

**Exercice 1.** Un premier problème concerne l'unicité de la représentation. En effet, deux listes différentes peuvent correspondre à un même polynôme. Par exemple,  $L = [0, 0, 1, 0]$  et  $M = [0, 0, 1]$  correspondent toutes les deux au polynôme  $X^2$ . De même, le polynôme nul peut être représenté par  $[]$  ou par  $[0]$ .

1. Écrire une fonction `polEgaux(L,M)` qui retourne `True` si  $L$  et  $M$  correspondent au même polynôme et `False` sinon.
2. Écrire une fonction `reduire(L)` qui supprime de  $L$  les 0 "inutiles" pour représenter  $P$ .
3. Une notion fondamentale sur les polynômes est le degré. Écrire une fonction `degre(P)` qui retourne le degré du polynôme  $P$ . Par convention, le degré du polynôme nul sera  $-1$ .

*Remarque.* En mathématiques, le degré du polynôme nul est  $-\infty$ . Mais il est plus facile, pour l'implémentation de le représenter ici par  $-1$ .

**Exercice 2.** Écrire une fonction `derive(P)` qui retourne une liste représentant  $P'$ .

**Exercice 3.** Écrire une fonction `somme(P,Q)` qui prend en argument deux listes représentant des polynômes et qui retourne la somme des deux.

**Exercice 4** (Méthode d'évaluation en un point). 1. On rappelle que pour évaluer en un point, on remplace  $X$  par valeur  $a$ . Écrire une fonction `evaluation(P,a)` qui retourne la valeur  $P(a)$ , le calcul étant effectué de manière naïve.

2. ( $\star$ ) Déterminer la complexité en nombre de multiplication de votre fonction.
3. Une des méthodes pour évaluer de manière efficace un polynôme en un point est appelée méthode de Horner. Elle consiste à remarquer le fait suivant :

$$a_0 + a_1X + a_2X^2 + a_nX^n = a_0 + X(a_1 + X(a_2 + \cdots + X(a_{n-1} + a_nX))) \cdots)$$

Pour calculer  $P(a)$ , on effectue alors les affectations suivantes :

- (a)  $S = a_n$ ,      (b)  $S = a_n a$ ,      (c)  $S = S + a_{n-1}$ ,      (d)  $S = Sa$ ,  
(e)  $S = S + a_{n-2}$ ,      (f)  $\cdots$       (g)  $S = S + a_1$ ,      (h)  $S = Sa$ ,      (i)  $S = S + a_0$ .

Par exemple, pour évaluer le polynôme  $X^3 + 5X^2 + 2X + 1$  en 2, on a donc successivement : 1, 2, 7, 14, 16, 32, 33. On a donc  $P(2) = 33$ .

Écrire une fonction `Horner(P,a)` qui calcule  $P(a)$  à l'aide de la méthode de Horner.

4. (★) Déterminer la complexité de la méthode de Horner en nombre de multiplications.
5. (Optionnel) Proposer un code qui permet de comparer informatiquement ces deux méthodes d'évaluation.

**Exercice 5** (Multiplication naïve). 1. Écrire une fonction `produit(P,Q)` prenant en argument deux listes représentant des polynômes et qui renvoie la liste représentant le produit correspondant.

2. (★) Expliciter la complexité en nombre de produits de nombres.

**Exercice 6** (Division naïve). 1. Écrire une fonction `division(P,Q)` prenant en argument deux listes représentant des polynômes et qui renvoie la liste représentant le reste du polynôme  $P$  par  $Q$ .

2. (★) Expliciter la complexité en nombre de produits et divisions de nombres.

## Pour les plus rapides

**Exercice 7.** On s'intéresse à un algorithme de multiplications de polynômes, appelé algorithme de Karatsuba.

Cet algorithme est basé sur la remarque suivante. Pour deux polynômes  $P = P_0 + X^n P_1$ ,  $Q = Q_0 + X^n Q_1$  avec  $P_0, P_1, Q_0, Q_1$  de degré au plus  $n-1$ , on a

$$PQ = P_0Q_0 + ((P_0 + P_1)(Q_0 + Q_1) - P_0Q_0 - P_1Q_1)X^n + X^{2n}P_1Q_1 \quad (1)$$

1. En exploitant cette égalité et en procédant de manière récursive, écrire une nouvelle fonction `produit_rapide(P,Q)` qui renvoie le produit de deux polynômes. On supposera pour simplifier que les listes représentant  $P$  et  $Q$  ont une taille qui est une puissance de 2.

*Remarque.* Pour des polynômes de petits degrés, on remarque qu'en pratique le produit naïf suffit et est même plus efficace. Ainsi, on peut proposer une fonction qui procède de manière naïve pour des petits degrés (par exemple 32) et qui utilise l'algorithme Karatsuba pour des degrés plus grand.

2. (★) Expliquer comment se ramener au cas des puissances de 2 dans le cas général.
3. (★) Proposer un code qui permet de comparer informatiquement ces deux méthodes d'évaluation.
4. (★) Déterminer la complexité de votre nouvelle fonction.

**Exercice 8** (Division quotient). Écrire une fonction `quotient_reste(P,Q)` prenant en arguments deux listes représentant des polynômes et qui renvoie un couple de listes  $(S, R)$  qui représente des polynômes vérifiant :  $P = SQ + R$ , où  $R$  est le reste et  $S$  le quotient.